Assignment 4 – Chia để Trị (Divide and Conquer)

Giới thiệu về Chia để Trị

Kỹ thuật **chia để trị** (Divide and Conquer) là một mô hình thiết kế giải thuật gồm ba bước chính:

1. **Divide (Chia):** Chia bài toán gốc thành một hoặc nhiều bài toán con nhỏ hơn có cùng cấu trúc.
2. **Conquer (Trị):** Giải đệ quy từng bài toán con. Nếu bài toán con đủ nhỏ (đạt đến trường hợp cơ sở), giải trực tiếp.
3. **Combine (Kết hợp):** Ghép lời giải của các bài toán con lại thành lời giải cho bài toán gốc.

Lược đồ tổng quát:

$$\text{DivideAndConquer}(P) = \begin{cases} \text{Giải trực tiếp} & \text{nếu } |P| \leq \text{ngưỡng} \\ \text{Chia} \to \text{Giải đệ quy} \to \text{Kết hợp} & \text{ngược lại} \end{cases}$$

Đặc điểm:

- Thường được biểu diễn bằng công thức truy hồi (recurrence relation).
- Hiệu quả khi chi phí kết hợp nhỏ hơn chi phí giải trực tiếp.
- Dễ song song hóa vì các bài toán con độc lập nhau.

Master Theorem

Xét công thức truy hồi có dạng:

$$T(n) = a\,T\left(\frac{n}{b}\right) + \Theta\left(n^k \log^p n\right)$$

với $a \geq 1$, $b > 1$, $k \geq 0$ và $p \in \mathbb{R}$.

1. Nếu $a > b^k$ thì $T(n) = \Theta\left(n^{\log_b a}\right)$
2. Nếu $a = b^k$ thì:

a) Nếu $p > -1$: $T(n) = \Theta\big(n^{\log_b a} \log^{p+1} n\big)$

b) Nếu $p = -1$: $T(n) = \Theta\big(n^{\log_b a} \log\log n\big)$

c) Nếu $p < -1$: $T(n) = \Theta\big(n^{\log_b a}\big)$

3. Nếu $a < b^k$ thì:

a) Nếu $p \geq 0$: $T(n) = \Theta\big(n^k \log^p n\big)$

b) Nếu $p < 0$: $T(n) = \Theta\big(n^k\big)$

Ý nghĩa trực quan: So sánh tốc độ tăng trưởng của a (số nhánh đệ quy) với b^k (tốc độ thu nhỏ bài toán nhân với chi phí kết hợp):

- $a > b^k$: chi phí thống trị bởi tầng lá – kết quả phụ thuộc $\log_b a$.
- $a = b^k$: chi phí đều ở mọi tầng – xuất hiện thêm nhân tử logarithm.
- $a < b^k$: chi phí thống trị bởi tầng gốc – kết quả phụ thuộc n^k.

Ví dụ minh họa

Ví dụ 1

Đề bài: Giải thuật A giải bài toán kích thước n bằng cách chia thành **5 bài toán con** kích thước $n/2$, giải đệ quy từng bài toán con, rồi kết hợp lời giải trong **thời gian tuyến tính**. Xác định độ phức tạp.

Lập công thức truy hồi:

$$T(n) = 5\,T\Big(\frac{n}{2}\Big) + \Theta(n), \qquad T(1) = \Theta(1).$$

Xác định tham số: $a = 5,\ b = 2,\ k = 1,\ p = 0.$
So sánh: $b^k = 2^1 = 2 < 5 = a \Rightarrow$ Áp dụng Trường hợp 1.
Kết quả:

$$T(n) = \Theta\big(n^{\log_2 5}\big) \approx \Theta\big(n^{2.322}\big)$$

Giải thích: Vì số nhánh đệ quy (5) tăng nhanh hơn tốc độ thu nhỏ bài toán ($2^1 = 2$), chi phí tính toán tập trung ở tầng lá của cây đệ quy, dẫn đến độ phức tạp phụ thuộc $n^{\log_2 5}$.

Ví dụ 2

Đề bài: Giải thuật B giải bài toán kích thước n bằng cách giải đệ quy **2 bài toán con** kích thước $n - 1$, rồi kết hợp trong **thời gian hằng số**. Xác định độ phức tạp.

Lập công thức truy hồi:

$$T(n) = 2\,T(n-1) + \Theta(1), \qquad T(1) = \Theta(1).$$

Lưu ý: Đây *không* phải dạng $T(n/b)$ nên không áp dụng trực tiếp Master Theorem. Khai triển trực tiếp:

$$\begin{aligned}
T(n) &= 2\,T(n-1) + c \\
&= 2\big(2\,T(n-2) + c\big) + c = 2^2 T(n-2) + 2c + c \\
&= 2^2\big(2\,T(n-3) + c\big) + 2c + c = 2^3 T(n-3) + c(2^2 + 2 + 1) \\
&\vdots \\
&= 2^{n-1} T(1) + c(2^{n-2} + \cdots + 2 + 1) = 2^{n-1} \cdot c_0 + c(2^{n-1} - 1).
\end{aligned}$$

Kết quả:

$$T(n) = \Theta(2^n)$$

Giải thích: Mỗi lần gọi đệ quy tạo ra 2 lời gọi, nên cây đệ quy có 2^{n-1} lá – độ phức tạp là hàm mũ, rất kém hiệu quả.

Ví dụ 3

Đề bài: Giải thuật C giải bài toán kích thước n bằng cách chia thành **9 bài toán con** kích thước $n/3$, giải đệ quy, rồi kết hợp trong $O(n^2)$. Xác định độ phức tạp.

Lập công thức truy hồi:

$$T(n) = 9\,T\left(\frac{n}{3}\right) + \Theta(n^2), \qquad T(1) = \Theta(1).$$

Xác định tham số: $a = 9,\ b = 3,\ k = 2,\ p = 0.$
So sánh: $b^k = 3^2 = 9 = a$ và $p = 0 > -1 \Rightarrow$ Áp dụng Trường hợp 2a.
Kết quả:

$$T(n) = \Theta\big(n^{\log_3 9} \cdot \log^{0+1} n\big) = \Theta\big(n^2 \log n\big)$$

Giải thích: Chi phí ở mỗi tầng của cây đệ quy là bằng nhau (n^2), và cây có $\log_3 n$ tầng, nên xuất hiện thêm nhân tử $\log n$.

Ví dụ 4 – Phân tích đoạn mã

Đề bài: Xét hàm Java sau, lập và giải công thức truy hồi.

Listing 1: Hàm đệ quy cần phân tích

```
1 public void function(int n) {
2     if (n > 1) {
3         System.out.println("*");   // O(1)
4         function(n / 2);            // goi de quy lan 1
5         function(n / 2);            // goi de quy lan 2
6     }
7 }
```

Phân tích từng dòng:

- Dòng 3: in một ký tự – $O(1)$.
- Dòng 4–5: hai lời gọi đệ quy với tham số $n/2$.

- Không có bước kết hợp – $O(1)$.

Công thức truy hồi:

$$T(n) = \begin{cases} \Theta(1) & \text{nếu } n \leq 1 \\ 2\,T(n/2) + \Theta(1) & \text{nếu } n > 1 \end{cases}$$

Xác định tham số: $a = 2,\ b = 2,\ k = 0,\ p = 0.$
So sánh: $b^k = 2^0 = 1 < 2 = a \Rightarrow$ Áp dụng Trường hợp 1.
Kết quả:

$$T(n) = \Theta\big(n^{\log_2 2}\big) = \Theta(n)$$

Kiểm tra trực quan: Cây đệ quy có $\log_2 n$ tầng, mỗi tầng i có 2^i nút, mỗi nút làm $O(1)$ việc. Tổng: $\sum_{i=0}^{\log_2 n} 2^i = 2n - 1 = \Theta(n)$. Khớp với kết quả trên.

Ví dụ 5 – Luyện tập Master Theorem

Áp dụng Master Theorem để xác định độ phức tạp các công thức sau:

a) $T(n) = 3\,T\left(\dfrac{n}{2}\right) + n^2$

$a = 3,\ b = 2,\ k = 2,\ p = 0.\quad b^k = 2^2 = 4 > 3 = a \quad \Rightarrow$ TH3a ($p = 0 \geq 0$):

$$T(n) = \Theta(n^k \log^p n) = \Theta(n^2)$$

Chi phí kết hợp n^2 thống trị – tầng gốc là nơi tốn công nhất.

b) $T(n) = 4\,T\left(\dfrac{n}{2}\right) + n^2$

$a = 4,\ b = 2,\ k = 2,\ p = 0.\quad b^k = 2^2 = 4 = a,\quad p = 0 > -1 \quad \Rightarrow$ TH2a:

$$T(n) = \Theta\big(n^{\log_2 4} \log^{0+1} n\big) = \Theta(n^2 \log n)$$

Chi phí cân bằng ở mọi tầng – xuất hiện thêm $\log n$.

c) $T(n) = 16\,T\left(\dfrac{n}{4}\right) + n$

$a = 16,\ b = 4,\ k = 1,\ p = 0.\quad b^k = 4^1 = 4 < 16 = a \quad \Rightarrow$ TH1:

$$T(n) = \Theta\big(n^{\log_4 16}\big) = \Theta\big(n^2\big)$$

Số nhánh đệ quy lớn – chi phí tập trung ở tầng lá.

d) $T(n) = 2\,T\left(\dfrac{n}{2}\right) + \dfrac{n}{\log n}$

$a = 2,\ b = 2,\ k = 1,\ p = -1.\quad b^k = 2 = a,\quad p = -1 \quad \Rightarrow$ TH2b:

$$T(n) = \Theta\big(n^{\log_2 2} \log\log n\big) = \Theta(n \log\log n)$$

e) $T(n) = \sqrt{2}\,T\left(\frac{n}{2}\right) + \log n$

$a = \sqrt{2},\ b = 2,\ k = 0,\ p = 1.\quad b^k = 2^0 = 1 < \sqrt{2} = a \quad \Rightarrow$ TH1:

$$T(n) = \Theta\left(n^{\log_2 \sqrt{2}}\right) = \Theta\left(n^{1/2}\right) = \Theta(\sqrt{n})$$

f) $T(n) = 3\,T\left(\frac{n}{3}\right) + \sqrt{n}$

$a = 3,\ b = 3,\ k = \frac{1}{2},\ p = 0.\quad b^k = 3^{1/2} = \sqrt{3} < 3 = a \quad \Rightarrow$ TH1:

$$T(n) = \Theta\left(n^{\log_3 3}\right) = \Theta(n)$$

Bài tập 1 – Công thức truy hồi đặc biệt

Xác định độ phức tạp của các công thức sau bằng phép đổi biến:

a) $T(n) = 2\,T(\sqrt{n}) + \log n$

Phương pháp đổi biến: Đặt $m = \log_2 n$, tức $n = 2^m$. Khi đó $\sqrt{n} = 2^{m/2}$. Đặt $S(m) = T(2^m)$:

$$S(m) = T(2^m) = 2\,T\left(2^{m/2}\right) + m = 2\,S(m/2) + m.$$

Áp dụng Master Theorem trên S: $a = 2,\ b = 2,\ k = 1,\ p = 0$.

$b^k = 2 = a$, $p = 0 > -1 \Rightarrow$ TH2a: $S(m) = \Theta(m \log m)$.

Thay ngược: $m = \log_2 n$:

$$T(n) = S(\log_2 n) = \Theta(\log n \cdot \log\log n)$$

b) $T(n) = T(\sqrt{n}) + 1$

Phương pháp đổi biến (tương tự ý a): Đặt $m = \log_2 n$, $S(m) = T(2^m)$:

$$S(m) = T(2^m) = T\left(2^{m/2}\right) + 1 = S(m/2) + 1.$$

Áp dụng Master Theorem trên S: $a = 1,\ b = 2,\ k = 0,\ p = 0$.

$b^k = 1 = a$, $p = 0 > -1 \Rightarrow$ TH2a: $S(m) = \Theta(\log m)$.

Thay ngược:

$$T(n) = S(\log_2 n) = \Theta(\log\log n)$$

Lưu ý: Có thể suy ý (b) từ ý (a): nếu $T(n) = 2\,T(\sqrt{n}) + \log n$ cho $\Theta(\log n \cdot \log\log n)$, thì $T(n) = T(\sqrt{n}) + 1$ (bài toán nhỏ hơn nhiều) tất nhiên cho kết quả nhỏ hơn nhiều: $\Theta(\log\log n)$.

Bài tập 2 – Cài đặt các bài toán Chia để Trị

Viết chương trình áp dụng kỹ thuật **chia để trị** để giải các bài toán sau. Với mỗi bài, đánh giá thời gian thực thi theo kích thước đầu vào (khuyến khích dùng biểu đồ).

1. **Min-Max:** Cho mảng số nguyên, tìm phần tử lớn nhất và nhỏ nhất. *Ý tưởng:* Chia đôi mảng, tìm min-max từng nửa, kết hợp. Độ phức tạp: $T(n) = 2T(n/2) + O(1) \Rightarrow \Theta(n)$, với số phép so sánh tối đa là $\lfloor 3n/2 \rfloor - 2$ (tối ưu hơn tuyến tính thông thường).

2. **Nhân ma trận:** Cho hai ma trận vuông A, B cấp n (n là lũy thừa của 2). Tính $A \times B$ bằng:

   - Chia để trị chuẩn: $T(n) = 8T(n/2) + \Theta(n^2) \Rightarrow \Theta(n^3)$.
   - **Strassen**: thay 8 phép nhân bằng 7, dùng thêm phép cộng: $T(n) = 7T(n/2) + \Theta(n^2) \Rightarrow \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$.

3. **QuickSort:** Chọn pivot, phân hoạch mảng, đệ quy hai phần. Trung bình: $\Theta(n \log n)$; xấu nhất: $\Theta(n^2)$.

4. **MergeSort:** Chia đôi, sắp xếp từng nửa, trộn hai nửa đã sắp. $T(n) = 2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$ mọi trường hợp.

5. **Majority Element:** Kiểm tra xem mảng đã sắp xếp có phần tử nào xuất hiện trên $\lfloor n/2 \rfloor$ lần không. Ý tưởng chia để trị: phần tử đa số của toàn mảng phải là phần tử đa số của ít nhất một nửa. Độ phức tạp: $\Theta(n \log n)$.

6. **Median của hai mảng:** Cho hai mảng đã sắp xếp kích thước m và n. Tìm trung vị của $m + n$ phần tử. Ý tưởng: loại bỏ $k/2$ phần tử mỗi bước (decrease-and-conquer). Độ phức tạp: $O(\log(\min(m, n)))$.

Bài tập 3 – Phát biểu và giải hai bài toán

Phát biểu **ít nhất hai bài toán**, phân tích, thiết kế giải thuật, đánh giá độ phức tạp và cài đặt chương trình minh họa kỹ thuật chia để trị hoặc các mô hình liên quan (giảm để trị, biến đổi để trị).

Bài toán 1: Bao lồi – Convex Hull (Sec. 5.5)

Phát biểu bài toán

Cho tập S gồm n điểm trên mặt phẳng. **Bao lồi** của S là đa giác lồi nhỏ nhất chứa toàn bộ các điểm trong S. Bài toán yêu cầu xác định các điểm là đỉnh của đa giác đó – gọi là **điểm cực** (extreme points).

Định nghĩa: Một điểm p của tập lồi được gọi là *điểm cực* nếu p không phải điểm giữa của bất kỳ đoạn thẳng nào có hai đầu mút thuộc tập đó.

Ví dụ: Cho 8 điểm $p_1, \ldots, p_8$. Bao lồi là đa giác với các đỉnh p_1, p_3, p_5, p_6, p_7 (các điểm cực). Các điểm còn lại nằm bên trong.

Quan sát nền tảng

Đoạn thẳng nối p_i và p_j là **cạnh của bao lồi** khi và chỉ khi **tất cả** các điểm còn lại của S nằm cùng một phía của đường thẳng qua p_i và p_j.

Cơ sở hình học

Đường thẳng qua (x_1, y_1) và (x_2, y_2) có phương trình $ax + by = c$ với:

$$a = y_2 - y_1, \qquad b = x_1 - x_2, \qquad c = x_1 y_2 - y_1 x_2.$$

Để kiểm tra các điểm có nằm cùng phía, ta tính dấu của $f(x, y) = ax + by - c$:

- $f(x, y) > 0$: điểm nằm phía trên/trái đường thẳng.
- $f(x, y) < 0$: điểm nằm phía dưới/phải đường thẳng.
- $f(x, y) = 0$: điểm nằm trên đường thẳng.

Nếu tất cả $f(x_k, y_k)$ cùng dấu (đều dương hoặc đều âm) thì $p_i p_j$ là cạnh của bao lồi.

Giải thuật vét cạn (Brute Force)

1. Với mỗi cặp điểm (p_i, p_j), $i \neq j$:
   a) Tính $a = y_j - y_i$, $b = x_i - x_j$, $c = x_i y_j - y_i x_j$.
   b) Với mọi điểm p_k $(k \neq i, j)$, tính $d_k = ax_k + by_k - c$.
   c) Nếu tất cả d_k cùng dấu $\Rightarrow p_i p_j$ là cạnh bao lồi.
2. Thu thập toàn bộ cạnh, liệt kê đỉnh theo thứ tự ngược chiều kim đồng hồ.

Listing 2: Convex Hull – Brute Force

```java
List<int[]> convexHull(int[][] pts) {
    int n = pts.length;
    List<int[]> edges = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            int a = pts[j][1] - pts[i][1]; // a = y2 - y1
            int b = pts[i][0] - pts[j][0]; // b = x1 - x2
            int c = pts[i][0]*pts[j][1] - pts[i][1]*pts[j][0];
            boolean allSameSide = true;
            int sign = 0;
            for (int k = 0; k < n; k++) {
                if (k == i || k == j) continue;
                int d = a*pts[k][0] + b*pts[k][1] - c;
                if (d != 0) {
                    if (sign == 0) sign = (d > 0) ? 1 : -1;
                    else if ((d > 0 ? 1 : -1) != sign) {
                        allSameSide = false; break;
                    }
```

```
                    }
                }
                if (allSameSide) edges.add(new int[]{i, j});
            }
        }
        return edges;
}
```

Phân tích độ phức tạp

- Số cặp điểm: $\dfrac{n(n-1)}{2} = O(n^2)$.
- Với mỗi cặp, kiểm tra $n-2$ điểm còn lại: $O(n)$.
- Tổng: $O(n^2) \times O(n) = O(n^3)$.

$$\Rightarrow \quad T(n) = O(n^3)$$

Giải thuật	Độ phức tạp	Ghi chú
Brute Force	$O(n^3)$	Bài này
Graham Scan	$O(n \log n)$	Sắp xếp + quét
QuickHull	$O(n \log n)$ TB	Chia để trị
Chan's Algorithm	$O(n \log h)$	h = số đỉnh bao lồi

Bài toán 2: Đồng xu giả – Fake Coin (Sec. 4.4)

Phát biểu bài toán

Trong n đồng xu trông giống hệt nhau, có **một đồng xu giả nhẹ hơn** các đồng xu thật. Dùng **cân hai đĩa** (balance scale) – chỉ cho biết đĩa nào nặng hơn hoặc hai đĩa cân bằng, không cho biết chênh lệch bao nhiêu – hãy tìm đồng xu giả với số lần cân ít nhất.

Giải thuật chia đôi (Decrease by factor 2)

Chia n đồng xu thành hai nhóm $\lfloor n/2 \rfloor$, để dư nếu n lẻ. Cân hai nhóm:

- **Hai nhóm bằng nhau:** đồng xu giả là đồng để dư.
- **Một nhóm nhẹ hơn:** đồng xu giả trong nhóm nhẹ, đệ quy.

Công thức truy hồi:

$$W(n) = W(\lfloor n/2 \rfloor) + 1, \quad W(1) = 0 \quad \Rightarrow \quad W(n) = \lfloor \log_2 n \rfloor$$

Giải thuật chia ba – tối ưu hơn (Decrease by factor 3)

Chia n đồng xu thành ba nhóm $\lfloor n/3 \rfloor$. Cân nhóm 1 và nhóm 2:

- **Bằng nhau:** đồng xu giả ở nhóm 3, đệ quy trên nhóm 3.
- **Nhóm 1 nhẹ hơn:** đệ quy trên nhóm 1.
- **Nhóm 2 nhẹ hơn:** đệ quy trên nhóm 2.

Công thức truy hồi:

$$W(n) = W(\lceil n/3 \rceil) + 1, \quad W(1) = 0 \quad \Rightarrow \quad W(n) = \lceil \log_3 n \rceil$$

Listing 3: Tìm đồng xu giả – chia ba

```
// Tra ve chi so cua dong xu gia trong mang coins[lo..hi]
int fakeCoin(int[] coins, int lo, int hi) {
    if (lo == hi) return lo; // chi con 1 dong, chinh la dong gia
    int size = hi - lo + 1;
    int third = size / 3;
    int g1lo = lo,            g1hi = lo + third - 1;
    int g2lo = lo + third,    g2hi = lo + 2*third - 1;
    int g3lo = lo + 2*third,  g3hi = hi;

    int w1 = sum(coins, g1lo, g1hi);
    int w2 = sum(coins, g2lo, g2hi);

    if (w1 < w2)          return fakeCoin(coins, g1lo, g1hi); // nhom 1 nhe hon
    else if (w2 < w1)     return fakeCoin(coins, g2lo, g2hi); // nhom 2 nhe hon
    else                  return fakeCoin(coins, g3lo, g3hi); // nhom 3 chua dong gia
}
```

So sánh hai chiến lược

n	Chia đôi: $\lfloor \log_2 n \rfloor$	Chia ba: $\lceil \log_3 n \rceil$	Tiết kiệm
9	3	2	1 lần cân
27	4	3	1 lần cân
81	6	4	2 lần cân
243	7	5	2 lần cân
729	9	6	3 lần cân

Chiến lược **chia ba luôn hiệu quả hơn hoặc bằng** chia đôi vì $\log_3 n < \log_2 n$ với mọi $n > 1$. Cả hai đều thuộc lớp $O(\log n)$ nhưng hằng số thực tế khác nhau.

Bài tập 4 – Bài toán giá chứng khoán (Stock Pricing)

Phát biểu bài toán

Cho mảng giá $P[1..n]$ của công ty CareerMonk.com trong n ngày liên tiếp, với $P[i]$ là giá cổ phiếu ngày thứ i. Giá không giống nhau ở mọi ngày. Có những ngày giá cao (nên bán) và những ngày giá thấp (nên mua).

Yêu cầu: Tìm hai chỉ số $i < j$ sao cho $P[j] - P[i]$ là **lớn nhất** (tức ngày mua i và ngày bán j cho lợi nhuận tối đa). Nếu không thể có lợi nhuận dương, trả về 0.

Ví dụ:

Ngày	1	2	3	4	5	6	7	8
Giá	10	7	5	8	11	9	3	6

Mua ngày 3 (giá 5), bán ngày 5 (giá 11) $\Rightarrow$ lợi nhuận tối đa $= 6$.

Quy về bài toán dãy con có tổng lớn nhất

Xét mảng hiệu $D[i] = P[i+1] - P[i]$ với $i = 1, \ldots, n-1$:

$$P[j] - P[i] = D[i] + D[i+1] + \cdots + D[j-1].$$

Tìm lợi nhuận tối đa $\equiv$ tìm **dãy con liên tiếp có tổng lớn nhất** (Maximum Subarray Sum) trên mảng D.

Với ví dụ trên: $D = [-3, -2, 3, 3, -2, -6, 3]$.

Dãy con tốt nhất: $D[3..4] = [3, 3]$, tổng $= 6$. Tương ứng mua ngày 3, bán ngày 5.

Giải thuật chia để trị

Chia mảng $D[lo..hi]$ tại mid. Dãy con tối ưu thuộc một trong ba trường hợp:

1. **Nằm hoàn toàn trong** $D[lo..mid]$: giải đệ quy bên trái.
2. **Nằm hoàn toàn trong** $D[mid+1..hi]$: giải đệ quy bên phải.
3. **Vắt qua** mid: tổng hậu tố lớn nhất của $D[lo..mid]$ cộng với tổng tiền tố lớn nhất của $D[mid+1..hi]$ – tính được trong $O(n)$.

Listing 4: Tìm lợi nhuận tối đa – chia để trị

```
// Tra ve [buyDay, sellDay, maxProfit]
int[] maxProfit(int[] P, int lo, int hi) {
    if (lo >= hi) return new int[]{lo, lo, 0};

    int mid = (lo + hi) / 2;
    int[] left  = maxProfit(P, lo, mid);
    int[] right = maxProfit(P, mid + 1, hi);

    // Tinh loi nhuan vuot qua ranh gioi
```

```
    int minPrice = P[mid], buyDay = mid;
    for (int i = mid - 1; i >= lo; i--)
        if (P[i] < minPrice) { minPrice = P[i]; buyDay = i; }

    int maxPrice = P[mid + 1], sellDay = mid + 1;
    for (int j = mid + 2; j <= hi; j++)
        if (P[j] > maxPrice) { maxPrice = P[j]; sellDay = j; }

    int crossProfit = maxPrice - minPrice;
    int[] cross = new int[]{buyDay, sellDay, crossProfit};

    // Tra ve phuong an co loi nhuan lon nhat
    if (left[2] >= right[2] && left[2] >= cross[2]) return
       left;
    if (right[2] >= left[2] && right[2] >= cross[2]) return
       right;
    return cross;
}
```

Phân tích độ phức tạp

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n), \quad T(1) = O(1).$$

$a = 2,\ b = 2,\ k = 1,\ p = 0.$ $b^k = 2 = a$, TH2a:

$$T(n) = O(n \log n).$$

Phương pháp	Độ phức tạp	Ghi chú
Vét cạn (xét mọi cặp i, j)	$O(n^2)$	Đơn giản nhất
Chia để trị (bài này)	$O(n \log n)$	Hiệu quả hơn
Kadane's Algorithm (DP)	$O(n)$	Tối ưu nhất

Tính đúng đắn

Mọi dãy con tối ưu đều rơi vào đúng một trong ba trường hợp (trái, phải, qua ranh giới). Giải thuật xét cả ba và chọn cái lớn nhất $\Rightarrow$ luôn cho kết quả đúng.

Bài tập 5 – Laptop không vỡ (Unbreakable Laptop)

Phát biểu bài toán

Ta làm việc trong tòa nhà n tầng. Muốn xác định **tầng trần** (ceiling): tầng thấp nhất mà khi thả laptop từ tầng đó trở lên sẽ bị vỡ. Nếu không tầng nào làm vỡ laptop thì ceiling $= n + 1$. Nếu tầng 1 đã làm vỡ thì ceiling $= 1$.

Ta có đúng **hai laptop**. Mỗi lần thả là một lần thử. Mục tiêu: **tối thiểu hóa số lần thử trong trường hợp xấu nhất** $f(n)$, với hy vọng $f(n)$ sublinear (tức $f(n) = o(n)$).

Tại sao không dùng tìm kiếm nhị phân?

Tìm kiếm nhị phân thả laptop ở tầng $n/2$ trước. Nếu vỡ, ta mất một laptop và chỉ còn một cái để quét tuyến tính từ tầng 1 đến $n/2 - 1 \Rightarrow$ trường hợp xấu nhất vẫn cần $O(n/2) = O(n)$ lần thử. Không đạt được sublinear.

Giải thuật với bước nhảy cố định

Thả laptop 1 ở các tầng $t, 2t, 3t, \ldots$ Khi vỡ ở bước kt, dùng laptop 2 quét từ $(k-1)t+1$ đến $kt - 1$ (tối đa $t - 1$ lần).

Số lần thử xấu nhất: $\lceil n/t \rceil + t - 1$. Tối thiểu khi $t = \sqrt{n}$:

$$f(n) \approx \sqrt{n} + \sqrt{n} = 2\sqrt{n} = O(\sqrt{n}).$$

Tuy nhiên, với t cố định, tổng số lần thử *không đều* theo vị trí của ceiling. Chiến lược tốt hơn: dùng bước nhảy *giảm dần*.

Giải thuật bước nhảy giảm dần – tối ưu

Ý tưởng: Chọn bước nhảy lần lượt $t, t-1, t-2, \ldots, 1$ sao cho tổng số lần thử *bằng nhau* bất kể ceiling ở đâu.

Thả laptop 1 ở các tầng: $t, \quad t + (t-1), \quad t + (t-1) + (t-2), \quad \ldots$

Khi laptop vỡ ở lần thử thứ k (đã dùng k lần thử), dùng laptop 2 quét tuyến tính trong $t - k$ tầng còn lại $\Rightarrow$ tổng: $k + (t - k) = t$ lần thử.

Điều kiện phủ hết n tầng:

$$t + (t-1) + (t-2) + \cdots + 1 \geq n \quad \Rightarrow \quad \frac{t(t+1)}{2} \geq n \quad \Rightarrow \quad t \geq \frac{-1 + \sqrt{1 + 8n}}{2}.$$

$$\Rightarrow \quad t_{\min} = \left\lceil \frac{-1 + \sqrt{1 + 8n}}{2} \right\rceil = \Theta(\sqrt{n}) \qquad \Rightarrow \quad f(n) = \Theta(\sqrt{n}).$$

Listing 5: Tìm tầng trần với 2 laptop – bước nhảy giảm dần

```
int findCeiling(int n) {
    // Tinh so lan thu toi da t
    int t = (int) Math.ceil((-1 + Math.sqrt(1 + 8.0 * n)) / 2);
    int floor = 0; // tang thap nhat chua kiem tra
    int step  = t; // buoc nhay hien tai

    // Pha 1: laptop 1, buoc nhay giam dan
    while (step >= 1) {
        int next = floor + step;
        if (next > n) next = n;
        if (breakOnDrop(next)) { // laptop vo
            // Pha 2: laptop 2, quet tuyen tinh
            for (int f = floor + 1; f < next; f++)
                if (breakOnDrop(f)) return f;
            return next;
        }
        floor = next;
```

```
        step--;
        if (floor >= n) return n + 1; // khong bao gio vo
    }
    return n + 1;
}
```

Minh họa với $n = 100$

$t = \lceil(-1 + \sqrt{801})/2\rceil = \lceil 13.65\rceil = 14$.

Thả laptop 1 tại các tầng: 14, 27, 39, 50, 60, 69, 77, 84, 90, 95, 99, 100.

Số lần thử xấu nhất: **14** lần (thay vì 100 lần nếu quét tuyến tính).

Phân tích độ phức tạp

Phương pháp	Số lần thử (xấu nhất)	Ghi chú
Tuyến tính (1 laptop)	n	Quét từ tầng 1
Nhị phân sai cách (2 laptop)	$\Theta(n)$	Pha 2 vẫn tuyến tính
Bước nhảy cố định $t = \sqrt{n}$	$\approx 2\sqrt{n}$	Không tối ưu hằng số
Bước nhảy giảm dần (bài này)	$\Theta(\sqrt{n})$	Tối ưu với 2 laptop

Bài tập 6 – Lát gạch hình chữ L (L-Shaped Tiling)

Phát biểu bài toán

Cho sàn vuông kích thước $2^n \times 2^n$ ô ($n \geq 1$). Một ô bất kỳ được chỉ định trước làm **ô thoát nước** (drainage cell) và không được lát gạch.

Một **gạch hình chữ L** (L-tromino) là mảnh gạch phủ đúng 3 ô tạo thành hình chữ L (gồm ô vuông 2×2 bỏ đi một góc). Có 4 hướng xoay của gạch L.

Yêu cầu: Chứng minh ta luôn có thể lát toàn bộ phần còn lại của sàn bằng các gạch hình chữ L, và thiết kế giải thuật thực hiện việc đó.

Kiểm tra tính khả thi: Tổng số ô cần lát là $4^n - 1$. Vì $4^n - 1 = (4-1)(4^{n-1} + \cdots + 1) = 3 \times \frac{4^n-1}{3}$, nên $4^n - 1$ chia hết cho 3 với mọi $n \geq 1 \Rightarrow$ số viên gạch cần dùng là nguyên.

Chứng minh và thiết kế giải thuật – Quy nạp

Bổ đề (mạnh hơn): Với mọi $n \geq 1$ và với *bất kỳ* vị trí nào của ô thoát nước, ta có thể lát sàn $2^n \times 2^n$ bằng các gạch L-tromino.

Chứng minh bằng quy nạp:

Cơ sở ($n = 1$): Sàn 2×2 gồm 4 ô, trừ đi 1 ô thoát nước còn 3 ô. Đặt đúng 1 viên gạch L vào 3 ô còn lại. Luôn thực hiện được vì gạch L phủ đúng 2×2 trừ một góc, và ô thoát nước ở góc nào đó.

Bước quy nạp ($n > 1$): Giả sử bổ đề đúng với $n - 1$. Ta chứng minh cho n:

1. **Chia:** Chia sàn $2^n \times 2^n$ thành 4 góc phần tư bằng nhau, mỗi góc kích thước $2^{n-1} \times 2^{n-1}$.

2. **Xác định góc đặc biệt:** Đúng một trong 4 góc chứa ô thoát nước – gọi là **góc đặc biệt**. Ba góc còn lại là **góc thường**.

3. **Đặt gạch trung tâm:** Đặt 1 viên gạch L tại vị trí trung tâm của sàn $2^n \times 2^n$, che đúng 3 ô nằm trong góc trong cùng của 3 góc thường. Kết quả:

   - Góc đặc biệt: có 1 ô bị loại (ô thoát nước).
   - Mỗi góc thường: có 1 ô bị loại (ô bị gạch trung tâm che).

   Mỗi trong 4 góc $2^{n-1} \times 2^{n-1}$ giờ có đúng **một ô bị loại**.

4. **Đệ quy:** Theo giả thuyết quy nạp, mỗi góc $2^{n-1} \times 2^{n-1}$ với 1 ô bị loại đều lát được. Áp dụng đệ quy cho cả 4 góc.

Minh họa với $n = 2$ (sàn 4×4)

Ô thoát nước ở góc dưới trái (hàng 4, cột 1). Gạch trung tâm che 3 ô ở vị trí (hàng 2, cột 2), (hàng 2, cột 3), (hàng 3, cột 2).

·	·	**C**	·
·	**C**	**C**	·
·	·	·	·
D	·	·	·

(D: ô thoát nước; C: ô bị gạch trung tâm che; các ô · được lát đệ quy ở 4 góc 2×2 riêng biệt.)

Pseudocode

Listing 6: Lát sàn L-tromino – chia để trị

```
int tileId = 1; // danh so moi vien gach

// tile: lat san board[r0..r0+size-1][c0..c0+size-1]
// (drainR, drainC): toa do o thoat nuoc (hoac o bi gan nhan)
void tile(int[][] board, int size, int r0, int c0,
          int drainR, int drainC) {
    if (size == 1) return; // co so: o thoat nuoc da danh dau

    int half = size / 2;
    int mid_r = r0 + half - 1;
    int mid_c = c0 + half - 1;
    int id = tileId++;

    // Dat gach L tai trung tam, phu 3 goc khong chua thoat
       nuoc
    // Goc tren-trai: [mid_r][mid_c]
    // Goc tren-phai: [mid_r][mid_c+1]
    // Goc duoi-trai: [mid_r+1][mid_c]
    // Goc duoi-phai: [mid_r+1][mid_c+1]

```

```java
    boolean inTopLeft     = drainR <= mid_r && drainC <= mid_c;
    boolean inTopRight    = drainR <= mid_r && drainC >  mid_c;
    boolean inBottomLeft = drainR >  mid_r && drainC <= mid_c;

    if (!inTopLeft)     board[mid_r  ][mid_c  ] = id; // phu goc TL
    if (!inTopRight)    board[mid_r  ][mid_c+1] = id; // phu goc TR
    if (!inBottomLeft) board[mid_r+1][mid_c  ] = id; // phu goc BL
    else                board[mid_r+1][mid_c+1] = id; // phu goc BR

    // De quy 4 goc phan tu
    // Goc tren-trai
    tile(board, half, r0, c0,
         inTopLeft ? drainR : mid_r,
         inTopLeft ? drainC : mid_c);
    // Goc tren-phai
    tile(board, half, r0, c0 + half,
         inTopRight ? drainR : mid_r,
         inTopRight ? drainC : mid_c + 1);
    // Goc duoi-trai
    tile(board, half, r0 + half, c0,
         inBottomLeft ? drainR : mid_r + 1,
         inBottomLeft ? drainC : mid_c);
    // Goc duoi-phai
    boolean inBottomRight = !inTopLeft && !inTopRight && !inBottomLeft;
    tile(board, half, r0 + half, c0 + half,
         inBottomRight ? drainR : mid_r + 1,
         inBottomRight ? drainC : mid_c + 1);
}
```

Phân tích độ phức tạp

Mỗi lần gọi đệ quy: đặt 1 gạch L ($O(1)$) rồi gọi đệ quy 4 lần với kích thước $n-1$:

$$T(n) = 4\,T(n-1) + O(1), \quad T(0) = O(1).$$

Khai triển: $T(n) = O(4^n)$.

Đây cũng chính là số viên gạch cần đặt: $(4^n - 1)/3 = O(4^n)$ viên, mỗi viên đặt trong $O(1)$ $\Rightarrow$ giải thuật **tối ưu về số phép toán** vì phải đặt mỗi viên gạch ít nhất một lần.

n	Kích thước sàn	Số viên gạch L cần dùng
1	2×2	1
2	4×4	5
3	8×8	21
4	16×16	85
5	32×32	341

Công thức tổng quát: số viên gạch $= (4^n - 1)/3$.